CONFIGURACIÓN PRINCIPAL DE TRABAJO

Neovim + Git Bash + latexmk + SumatraPDF

1. Compilación continua y tiempos

Desde Git Bash, situado en la carpeta del proyecto, ejecutar una sola vez:

```
latexmk -xelatex -synctex=1 -pvc -time general.tex
```

Dejar Git Bash abierto. Al guardar en Neovim con :w, latexmk detecta
los cambios, recompila y SumatraPDF actualiza el PDF abierto.
La opción -time muestra los tiempos de las ejecuciones del compilador.
Para detener la vigilancia: Ctrl+C en Git Bash.

2. Compilar solamente el capítulo de trabajo

En el preámbulo de general.tex, antes de \begin{document}:

```
\includeonly{cap-3}
```

Esto funciona si el capítulo está incorporado con \include{cap-3}
(no con \input{cap-3}). LaTeX conserva la numeración de capítulos
y puede recuperar referencias mediante los archivos auxiliares previos.

3. Volver a compilar el documento completo

```
% \includeonly{cap-3}
```

Agregar % al inicio de la línea, guardar con :w y esperar la compilación.
Si los capítulos están dentro de una subcarpeta, escribir la ruta en
\includeonly tal como aparece en sus correspondientes \include.

Secuencia de uso

Editar en Neovim → guardar con :w → latexmk recompila
→ SumatraPDF muestra el resultado → Git Bash sigue vigilando.

NEOVIM + LaTeX

Guía práctica para Windows · edición, compilación y visualización de PDF

Entorno recomendado: Neovim para editar, Git Bash para compilar con latexmk y SumatraPDF para visualizar sin cerrar el archivo. TeXworks y LuaLaTeX se presentan como alternativa.

Contenido: flujo de trabajo · apertura y edición · navegación · búsqueda · archivos · VimTeX · compilación y Biber · medición de tiempos · instalación · init.lua · SyncTeX · TeXworks · referencia rápida.

1. Flujo de trabajo recomendado

Abrir general.tex en Neovim, mantener general.pdf abierto en SumatraPDF, y ejecutar desde Git Bash (situado en la carpeta del proyecto):

```
latexmk -xelatex -synctex=1 general.tex
```

Luego de modificar archivos, guardar con :w. El PDF se actualiza en SumatraPDF cuando termina la compilación. Para no lanzar el comando cada vez, utilizar el modo continuo:

```
latexmk -xelatex -synctex=1 -pvc general.tex
```

Con -pvc, latexmk observa cambios y recompila automáticamente. Detener con Ctrl+C. Evitar dos procesos de compilación continua simultáneos (por ejemplo Git Bash y VimTeX).

Bibliografía con Biber

Si el proyecto utiliza biblatex con backend=biber, latexmk normalmente invoca Biber cuando es necesario. Si se requiere una intervención manual:

```
latexmk -xelatex general.tex
biber general
latexmk -xelatex general.tex
```

No hay que ejecutar siempre los tres comandos: en condiciones normales alcanza con latexmk.

2. Abrir Neovim y ubicar el proyecto

En PowerShell

```
cd F:\LATEX\neovim
nvim general.tex
```

O abrir directamente:

```
nvim F:\LATEX\neovim\general.tex
```

Dentro de Neovim (modo NORMAL)

Comando	Acción
:pwd	Mostrar directorio actual
:cd F:/LATEX/neovim	Cambiar al directorio del proyecto
:e general.tex	Abrir archivo
:Ex	Explorar archivos de la carpeta

En comandos internos de Neovim se recomienda utilizar barras / para las rutas de Windows.

3. Modos y edición esencial

NORMAL: navegar y ejecutar comandos. INSERT: escribir texto. VISUAL: seleccionar. Para volver a NORMAL desde INSERT, pulsar Esc.

Tecla	Acción
i / a	Insertar antes / después del cursor
o / O	Crear línea debajo / arriba
I / A	Insertar al principio / final de línea
s	Sustituir carácter y comenzar inserción
Esc	Volver a NORMAL
u / Ctrl+r	Deshacer / rehacer
dd / yy / p / P	Eliminar línea / copiar línea / pegar después / antes
x / cw / cc	Eliminar carácter / cambiar palabra / cambiar línea
.	Repetir la última modificación

4. Guardar, salir y renombrar

Comando	Acción
:w	Guardar archivo
:q	Cerrar (si no hay cambios pendientes)
:wq	Guardar y salir
:x	Guardar si hubo cambios y salir
:q!	Salir sin guardar
:wa	Guardar todos los buffers modificados
:qa	Salir de todas las ventanas
:w test.tex	Escribir una copia en test.tex (no cambia el archivo activo)
:saveas test.tex	Guardar con otro nombre y continuar editándolo

5. Buscar y reemplazar

Comando	Acción
/palabra	Buscar hacia adelante (Enter confirma)
?palabra	Buscar hacia atrás
n / N	Coincidencia siguiente / dirección contraria
* / #	Buscar palabra bajo el cursor hacia adelante / atrás
:%s/viejo/nuevo/g	Reemplazar todas las coincidencias del archivo
:%s/viejo/nuevo/gc	Reemplazar solicitando confirmación
:s/viejo/nuevo/g	Reemplazar en la línea actual

6. Desplazamiento rápido

Tecla	Acción
Ctrl+d / Ctrl+u	Bajar / subir media pantalla
Ctrl+f / Ctrl+b	Bajar / subir una pantalla
gg / G	Inicio / final del archivo
15G / :15	Ir a la línea 15
0 / ^ / $	Inicio absoluto / primer carácter / final de la línea
w / b	Una palabra adelante / atrás
{ / }	Párrafo anterior / siguiente
%	Saltar al delimitador correspondiente

7. Trabajar con varios archivos y ventanas

```
:e secciones/cap-4.tex
:split secciones/cap-4.tex
:vsplit secciones/cap-4.tex
```

Tecla o comando	Acción
Ctrl+w w	Cambiar a otra ventana
Ctrl+w h / l	Ventana izquierda / derecha
Ctrl+w j / k	Ventana inferior / superior
Ctrl+w q	Cerrar ventana actual
:ls	Listar buffers
:bn / :bp	Buffer siguiente / anterior
:b nombre	Cambiar al buffer indicado

8. Archivo principal del proyecto LaTeX

En un proyecto con general.tex en la raíz y capítulos en secciones/, agregar al inicio de cada capítulo:

```
% !TeX root = ../general.tex
```

Si está en la misma carpeta que el principal:

```
% !TeX root = general.tex
```

La ruta es relativa al archivo que contiene la directiva. Verificar el archivo principal detectado con :VimtexInfo. Si se modifica esta directiva y no se actualiza la detección, probar :VimtexReload.

9. Compilar con VimTeX

VimTeX requiere estar instalado y configurado. Los siguientes atajos suponen que se conserva el líder local predeterminado \.

Comando	Acción
\ll	Iniciar o alternar compilación continua
\lv / :VimtexView	Visualizar PDF
\lk / :VimtexStop	Detener compilador
\le	Ver errores
\lc / :VimtexClean	Limpiar archivos auxiliares
:VimtexCompile	Iniciar compilación
:VimtexInfo	Mostrar información del proyecto

Si hay problemas persistentes con archivos auxiliares:

```
:VimtexStop
:VimtexClean
:VimtexCompile
```

Limpiar no debe formar parte de cada compilación. Una opción sin VimTeX es :!latexmk -xelatex general.tex. Evitar :!latexmk -xelatex % si se está editando un capítulo: % representa el archivo actual.

10. Medir tiempos de compilación

En Git Bash sí está disponible el comando time:

```
time xelatex general.tex
time xelatex -synctex=0 general.tex
time xelatex -interaction=nonstopmode -halt-on-error -synctex=0 general.tex
time xelatex -draftmode general.tex
```

La opción -draftmode no genera el PDF final. Para medir el ciclo completo, incluida la bibliografía si corresponde:

```
time latexmk -xelatex general.tex
time latexmk -gg -xelatex general.tex
```

-gg obliga a reconstruir incluso cuando latexmk considera el proyecto actualizado. En PowerShell, medir mediante:

```
Measure-Command { latexmk -xelatex general.tex }
```

11. Instalación y comprobación

Instalar MiKTeX, Neovim, Git y SumatraPDF. Para la compilación con latexmk, disponer también de Perl. Verificar desde la terminal:

```
nvim --version
git --version
xelatex --version
lualatex --version
latexmk -v
biber --version
```

La configuración de Neovim se guarda habitualmente en:

```
C:\Users\TU_USUARIO\AppData\Local\nvim\init.lua
```

Desde PowerShell, abrirlo con nvim "$env:LOCALAPPDATA\nvim\init.lua"; crear la carpeta si falta.

12. Configuración mínima de init.lua

Una única configuración con lazy.nvim, VimTeX, latexmk, XeLaTeX y SumatraPDF:

```
vim.g.maplocalleader = "\\"

local lazypath = vim.fn.stdpath("data") .. "/lazy/lazy.nvim"
if not vim.uv.fs_stat(lazypath) then
  vim.fn.system({
    "git", "clone", "--filter=blob:none",
    "https://github.com/folke/lazy.nvim.git", lazypath,
  })
end
vim.opt.rtp:prepend(lazypath)

require("lazy").setup({
  {
    "lervag/vimtex",
    lazy = false,
    init = function()
      vim.g.vimtex_view_method = "sumatrapdf"
      vim.g.vimtex_compiler_method = "latexmk"
      vim.g.vimtex_compiler_latexmk = {
        executable = "latexmk",
        options = {
          "-xelatex",
          "-synctex=1",
          "-interaction=nonstopmode",
          "-file-line-error",
        },
      }
    end,
  },
})
```

Guardar y reiniciar Neovim. Usar :Lazy para consultar plugins y :VimtexInfo para comprobar VimTeX. Si no se encuentra SumatraPDF, verificar su instalación y ruta.

13. Sincronización con SumatraPDF

Activar SyncTeX al compilar:

```
latexmk -xelatex -synctex=1 general.tex
```

Se genera general.synctex.gz. En VimTeX, :VimtexView o \lv permite abrir/enfocar el visor y usar búsqueda directa. Para buscar desde SumatraPDF hacia una instancia particular de Neovim se necesita configurar adicionalmente la búsqueda inversa; -synctex=1 no basta por sí solo.

14. Alternativa: TeXworks + LuaLaTeX

Abrir general.tex en TeXworks, seleccionar LuaLaTeX y compilar. El visor integrado permite consultar el PDF. Para proyectos con bibliografía, el ciclo automatizado es:

```
latexmk -lualatex general.tex
```

XeLaTeX y LuaLaTeX admiten fuentes OpenType y fontspec, pero no siempre producen resultados idénticos. Conviene elegir un motor estable para cada proyecto y verificar compatibilidad antes de cambiarlo.

15. Resumen de consulta inmediata

Necesito	Comando
Abrir documento	nvim general.tex
Insertar / volver a NORMAL	i / Esc
Guardar / salir	:w / :wq
Buscar / reemplazar	/texto / :%s/viejo/nuevo/g
Inicio / fin	gg / G
Abrir explorador	:Ex
Ver archivo principal	:VimtexInfo
Compilar	latexmk -xelatex -synctex=1 general.tex
Compilar automáticamente	latexmk -xelatex -synctex=1 -pvc general.tex
Ver PDF con VimTeX	\lv
Detener latexmk -pvc	Ctrl+C

Regla operativa: editar en Neovim → guardar con :w → compilar con latexmk (o mantener -pvc activo) → visualizar el PDF actualizado en SumatraPDF.

MODO VISUAL: selección y edición

Complemento de la guía. En Neovim, NORMAL permite desplazarse y ejecutar órdenes; VISUAL permite seleccionar un fragmento antes de actuar; INSERT permite escribir.

Entrar y salir de VISUAL

Tecla en NORMAL	Acción
v	Selección carácter por carácter
V	Seleccionar líneas completas
Ctrl+v	Selección rectangular o por columnas
Esc	Cancelar la selección y volver a NORMAL

Operaciones sobre una selección

Tecla	Efecto
y	Copiar la selección
d	Eliminar o cortar selección
p	Reemplazar selección con texto copiado (en VISUAL)
>	Aumentar sangría
<	Disminuir sangría
=	Ajustar automáticamente la sangría

Ejemplo: copiar tres líneas de LaTeX

Presionar Esc, posicionarse en la primera línea, pulsar V, bajar dos líneas con j, pulsar y, desplazarse al destino y pulsar p.

Comandos relacionados (modo NORMAL)

Comando	Acción
u / Ctrl+r	Deshacer / rehacer
dd / yy / p	Eliminar línea / copiar línea / pegar
:ls / :b#	Ver buffers / volver al buffer anterior
/texto / n / N	Buscar / siguiente / coincidencia contraria

Nota: al usar Ctrl+v en Windows Terminal puede intervenir el pegado de la terminal. Si eso ocurre, desde NORMAL se puede entrar a Visual Block mediante Ctrl+q en muchas configuraciones.

UNOKAI: indicador fuerte de INSERT

Configuración actual elegida: Unokai con barra de estado roja en INSERT, verde en NORMAL y azul en VISUAL. El indicador clásico -- INSERT -- permanece visible debajo de la barra.

Abrir la configuración

```
:e $LOCALAPPDATA/nvim/init.lua
```

Agregar el siguiente bloque al final del archivo existente. No reemplazar la configuración de lazy.nvim ni VimTeX. Si ya hay un bloque anterior que modifica StatusLine según el modo, sustituirlo para evitar duplicaciones.

```
vim.cmd.colorscheme("unokai")
vim.opt.showmode = true       -- muestra -- INSERT --
vim.opt.laststatus = 2
vim.opt.statusline = " %{mode()} %f %m %=%l:%c "

local colores_modo = {
  n = "#286B45",  -- NORMAL: verde
  i = "#D00000",  -- INSERT: rojo
  v = "#2463C5",  -- VISUAL: azul
  V = "#2463C5",
}

local function actualizar_barra()
  local modo = vim.api.nvim_get_mode().mode:sub(1, 1)
  vim.api.nvim_set_hl(0, "StatusLine", {
    fg = "#FFFFFF",
    bg = colores_modo[modo] or "#505050",
    bold = true,
  })
end

vim.api.nvim_create_autocmd({
  "ModeChanged", "VimEnter", "ColorScheme"
}, { callback = actualizar_barra })
actualizar_barra()
```

Guardar y aplicar

Presionar Esc, escribir :w, cerrar y abrir Neovim. Para revisar el tema activo: :colorscheme.

Si aparece el error «module modo_barra not found»: buscar /modo_barra en init.lua y eliminar la línea anterior que invoque require('modo_barra'); este bloque no utiliza ese módulo.

Nota: la barra roja corresponde a StatusLine (la línea con el nombre del archivo); no cambia el fondo del área donde se escribe LaTeX.

ANEXO · Sincronización bidireccional

Configuración comprobada: Neovim + VimTeX + LuaLaTeX/latexmk + SumatraPDF. F5 localiza en el PDF la línea del editor; doble clic en SumatraPDF abre el archivo .tex y posiciona el cursor en Neovim. Se usa un servidor fijo y un pequeño script de PowerShell.

1. Compilación continua con SyncTeX

```
latexmk -lualatex -synctex=1 -pvc -time general.tex
```

Ejecutar en Git Bash desde la carpeta del proyecto. Guardar con :w en Neovim para recompilar y actualizar SumatraPDF. Detener con Ctrl+C. Para trabajar con un capítulo individual, activar \includeonly{cap-3} en el preámbulo del archivo general.tex (requiere capítulos incorporados mediante \include).

2. De Neovim a SumatraPDF (F5)

En el archivo init.lua (abrirlo con :e $LOCALAPPDATA/nvim/init.lua), agregar este atajo sin reemplazar la configuración existente:

```
vim.keymap.set("n", "<F5>", "<cmd>VimtexView<CR>", {
  desc = "Sincronizar Neovim con SumatraPDF",
  silent = true,
})
```

Ubicar el cursor en el texto y presionar Esc y F5. La orden equivalente es :VimtexView. Es necesario compilar con -synctex=1.

3. Servidor estable para recibir el doble clic

Agregar al final de init.lua esta instrucción. Reiniciar Neovim después de guardar. Solo una instancia debe usar esta dirección fija.

```
vim.fn.serverstart("\\\\.\\pipe\\nvim-sumatra")
```

Verificar desde PowerShell, con Neovim abierto:

```
nvr --nostart --servername '\\.\pipe\nvim-sumatra' -c 'echo getcwd()'
```

La dirección indicada en :echo v:servername puede tener un número variable; eso es normal. El servidor fijo adicional es el que utiliza SumatraPDF.

ANEXO · Configuración de búsqueda inversa

4. Instalar Neovim Remote (nvr)

```
python --version
python -m pip install neovim-remote
nvr --version
```

En la instalación probada, Python 3.14.3 y nvr 2.5.1. Si nvr no se reconoce, agregar al PATH de usuario esta carpeta y abrir una nueva terminal:

```
C:\Users\moran\AppData\Local\Python\pythoncore-3.14-64\Scripts
```

La versión utilizada de nvr presentó una advertencia por pkg_resources. Para mantener compatibilidad en ese entorno se instaló setuptools<81:

```
python -m pip install "setuptools<81"
```

5. Script PowerShell que recibió los datos del PDF

Archivo guardado como C:\Users\moran\sumatra-neovim.ps1. Esta es la configuración que se probó correctamente para abrir cap-4.tex en la línea 100:

```
param(
    [string]$Archivo,
    [int]$Linea
)

$nvr = "$env:LOCALAPPDATA\Python\pythoncore-3.14-64\Scripts\nvr.exe"
$servidor = '\\.\pipe\nvim-sumatra'
$ruta = $Archivo.Replace('\', '/')

& $nvr --nostart --servername $servidor `
    -c "edit $ruta" `
    -c "$Linea"
```

Para probarlo desde PowerShell sin cambiar la política de ejecución permanente:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File `
  "$HOME\sumatra-neovim.ps1" "F:\LATEX\neovim\cap-4.tex" 100
```

Nota: el script anterior reproduce la solución efectivamente probada. Si se usan carpetas o nombres de archivo con espacios o caracteres especiales, conviene adaptar el tratamiento de rutas antes de reutilizarlo.

6. Configuración dentro de SumatraPDF

Abrir Configuración → Opciones → Comando de búsqueda inversa y reemplazar la configuración anterior de VS Code por:

```
powershell.exe -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass
-File "C:\Users\moran\sumatra-neovim.ps1" "%f" %l
```

%f contiene la ruta del archivo fuente y %l su número de línea. Hacer doble clic en un párrafo del PDF: SumatraPDF transmite ambos datos y Neovim salta al archivo y posición correspondientes. El visor y Neovim deben estar abiertos.

ANEXO · Verificación y problemas frecuentes

7. Pruebas concretas que funcionaron

```
nvr --nostart --servername '\\.\pipe\nvim-sumatra' `
  -c 'edit F:/LATEX/neovim/cap-4.tex' -c '100'
```

La prueba abrió cap-4.tex y movió el cursor hasta la línea 100. Posteriormente, el script y el doble clic en SumatraPDF funcionaron correctamente.

8. Si vuelve a fallar

Se abre una terminal adicional: comprobar que el comando de búsqueda inversa apunte al script de PowerShell y use -WindowStyle Hidden y -NoProfile.

No encuentra el servidor: comprobar que Neovim siga abierto y que haya arrancado el servidor fijo. Utilizar --nostart en las pruebas para impedir que nvr cree otra instancia.

E325 / archivo .swp: indica otra sesión sobre el mismo archivo o un cierre inesperado. Revisar si existen instancias de Neovim abiertas; recuperar cambios con R si es necesario y guardar antes de eliminar archivos swap. Evitar dos procesos editando el mismo .tex.

Advertencia E45 readonly: verificar si se abrió el archivo en solo lectura a raíz de un swap o por permisos. No forzar el guardado sin comprobar cuál es la copia activa.

La búsqueda inversa no mueve el cursor: comprobar que el PDF corresponda a la última compilación y que exista el archivo general.synctex.gz. La sincronización requiere que la compilación no use -synctex=0.

9. Resumen del flujo habitual

Acción	Cómo hacerlo
Editar y guardar	Neovim: i → texto → Esc → :w
Compilar automáticamente	Git Bash: latexmk -lualatex -synctex=1 -pvc -time general.tex
Neovim → PDF	F5 (VimtexView)
PDF → Neovim	Doble clic en SumatraPDF (script + nvr)
Detener compilación	Ctrl+C en Git Bash

ANEXO · Lazygit integrado con Neovim

Actualización: octubre de 2026. Complementa la guía de Neovim, VimTeX y SumatraPDF sin modificar las secciones anteriores.

1. Qué es Lazygit

Lazygit es una interfaz de terminal para Git. Permite revisar archivos, preparar cambios (**staging**), registrar commits, consultar versiones y sincronizar con GitHub. No reemplaza Git ni es un repositorio independiente.

2. Instalación en Windows

Desde PowerShell, si todavía no está instalado:

```
winget install --id JesseDuffield.lazygit -e
```

Comprobar desde Git Bash:

```
lazygit --version
```

3. Integración con lazy.nvim

Dentro de la lista de plugins de **require("lazy").setup({ ... })** en **init.lua**, incorporar esta entrada (sin reemplazar las existentes):

```
{
"kdheepak/lazygit.nvim",
cmd = "LazyGit",
dependencies = { "nvim-lua/plenary.nvim" },
keys = { { "<leader>gg", "<cmd>LazyGit<CR>", desc = "Abrir Lazygit" } },
},
```

Antes de inicializar lazy.nvim, definir la tecla líder:

```
vim.g.mapleader = " "
```

Guardar **init.lua**, ejecutar **:Lazy sync** y reiniciar Neovim. El comando **:LazyGit** lo abre; con líder Espacio también funciona **Espacio, g, g** (teclas consecutivas).

4. Paneles principales

Panel	Qué muestra
1 · Status	Repositorio y rama actual
2 · Files	Archivos nuevos/modificados y su preparación
3 · Local branches	Ramas, remotos y etiquetas
4 · Commits	Historial de commits
5 · Stash	Cambios apartados temporalmente

ANEXO · Uso diario de Lazygit

5. Atajos esenciales

Tecla / acción	Resultado
Espacio, g, g (en Neovim)	Abrir Lazygit mediante el atajo configurado
1 a 5	Cambiar al panel correspondiente
Flechas o j / k	Desplazarse por las entradas
Espacio (panel Files)	Preparar o retirar el archivo seleccionado
a (panel Files)	Alternar preparación de los archivos
c (panel Files)	Iniciar un commit
p / P (panel Branches)	Push / Pull, según contexto y configuración
?	Ver mapa de teclas disponible en el panel actual
q	Cerrar Lazygit o volver a la vista anterior

6. Ejemplo: registrar cambios en un capítulo LaTeX

1. Guardar el capítulo con **:w**. **2.** Abrir Lazygit con **Espacio, g, g**. **3.** En Files, revisar los archivos y usar **Espacio** en aquellos que quieras incluir. **4.** Comprobar el diff de Staged changes. **5.** Presionar **c**, escribir un mensaje descriptivo y confirmar. **6.** Si corresponde y existe un remoto, hacer push para publicar el commit en GitHub.

7. Qué significan las marcas

??: archivo sin seguimiento (untracked). **A**: archivo nuevo agregado al índice (staged). **M**: archivo modificado; su ubicación y color permiten distinguir si está preparado o pendiente. El panel **Staged changes** muestra exactamente lo que entrará al próximo commit.

8. Seguridad y comprobaciones

Un **commit** es local: no se publica hasta hacer **push**. Los comandos **Discard**, **Reset** y **Hard reset** pueden destruir cambios o mover referencias del historial. No usarlos sin revisar su alcance. En Git Bash, **git status** y **git remote -v** ayudan a identificar el repositorio y su remoto.

Nota: Lazygit actúa sobre el repositorio Git correspondiente al directorio de trabajo; al abrir proyectos distintos, verificar el nombre y rama en Status. Los archivos auxiliares de LaTeX deben excluirse con un **.gitignore** apropiado.

ANEXO · Lazygit en Neovim: práctica real

Ampliación de octubre de 2026. Ejemplo probado: proyecto F:/LATEX/neovim, rama master, Git Bash en Windows. Conserva la guía anterior y explica los pasos que surgieron al trabajar con commits y GitHub.

1. Abrir Lazygit y reconocer los paneles

En modo normal de Neovim: **Espacio, g, g** (teclas sucesivas). Alternativa: **:LazyGit**. Para salir de Lazygit: **q**. Si el atajo no responde, pulsar Esc y comprobar que no estás escribiendo en un terminal o cuadro de diálogo.

Panel	Qué muestra
[1] Status	Nombre y estado general del repositorio.
[2] Files	Archivos modificados, nuevos y preparados.
[3] Local branches / Remotes	Rama master, ramas remotas y sincronización.
[4] Commits / Reflog	Historial de commits, mensajes y cambios registrados.
[5] Stash	Cambios guardados provisionalmente.

2. Cambios y staging: cómo leer el estado

?? = archivo no seguido; **A** = agregado; **M** = modificado; **MM** = el mismo archivo tiene parte preparada y parte todavía sin preparar. **Unstaged changes** indica cambios pendientes; **Staged changes** son los que entrarían al próximo commit.

En el panel Files, **Espacio** prepara o retira un archivo. La vista Diff marca en verde lo agregado y en rojo lo eliminado. Revisar siempre el panel Staged antes de confirmar el commit.

3. Crear un commit normal

Guardar el .tex con :w. Abrir Lazygit; en Files, seleccionar los archivos y pulsar Espacio. Pulsar **c**, escribir un mensaje descriptivo en Commit summary y confirmar según el cuadro mostrado. En Commits aparecerá el nuevo registro; los cambios sin preparar permanecen en los archivos.

Si se abre Vim para escribir el mensaje: para cancelar el commit sin perder cambios preparados, usar Esc, escribir **:cq** y Enter. Para guardar el mensaje y salir de Vim, usar Esc, **:wq** y Enter.

4. Preparar sólo un fragmento de un archivo

Caso probado: cap-5.tex tenía dos cambios distintos en dos párrafos. En Files seleccionar cap-5.tex y entrar a su vista de diferencias con Enter. El panel presenta Unstaged changes y Staged changes; la barra inferior informa las teclas activas (por ejemplo, **Stage: Space** y **Select line-by-line: a**).

En la vista de cambios, ubicar el bloque o las líneas que se desean preparar y aplicar Espacio; si se necesita selección línea por línea, usar la tecla **a** que indica esa pantalla y elegir las líneas. Usar Tab para alternar entre los dos paneles. Si hay algo preparado por error, retirarlo del staging antes de confirmar.

Resultado: el commit guarda **sólo la parte preparada**. El resto continúa como Unstaged, en el archivo físico, sin perderse. Antes de pulsar c, comprobar si también hay otros archivos en staging.

5. Publicar en GitHub por primera vez

Primero debe existir un repositorio remoto en GitHub. Se puede crear desde Git Bash, situado en la raíz del proyecto, con GitHub CLI si el nombre está disponible:

```
gh auth status
cd /f/LATEX/neovim
git remote -v
gh repo create neovim --private --source=. --remote=origin
```

No crear otro repositorio si ya existe origin: comprobar primero git remote -v. La creación anterior establece la conexión, pero no publica commits hasta hacer push.

6. Push inicial y upstream

En Lazygit se probó el panel [3] Local branches, sobre master. Al intentar asignar **origin master** apareció: «upstream branch origin/master not found». Era esperable porque todavía no existía la rama remota. El propio aviso indicó usar **Shift+P** para publicar por primera vez.

Con la configuración probada, **Shift+P** hizo el primer push. Como alternativa clara y equivalente desde Git Bash:

```
git push -u origin master
```

Esto publica la rama y registra que master sigue a origin/master. Después se observó «branch master set up to track origin/master» y el historial mostraba **HEAD → master, origin/master** sobre el mismo commit.

7. Verificar la sincronización

```
git status -sb
# Resultado observado:
## master...origin/master
```

Sin «ahead» ni «behind», las ramas local y remota coinciden **según la última información remota conocida**. Si querés actualizar esa información antes de verificar, ejecutá git fetch. Si además no hay archivos listados, el directorio está limpio.

Para abrir el proyecto remoto desde la carpeta del repositorio:

```
gh repo view --web
```

8. Atajos y precauciones

Acción	Tecla o comando
Abrir Lazygit	Espacio, g, g; o :LazyGit
Preparar/retirar archivo	Espacio en Files
Commit	c en Files
Ver atajos disponibles	? (dependen del panel)
Push inicial probado	Shift+P en la pantalla de ramas
Salir de Lazygit	q
Comprobar Git	git status -sb

Atención: los atajos p / P dependen del contexto y del mapa de teclas; consultar **?** antes de utilizarlos. No confundir Reset o Discard con retirar del staging: esas funciones pueden eliminar cambios. También conviene usar .gitignore para excluir auxiliares LaTeX y copias temporales *~; si ya entraron en un commit, ignorarlos después no los elimina del historial.

Anexo: Git y Lazygit en Neovim

Conceptos y procedimientos probados en el repositorio neovim, Lazygit 0.66.1, Windows y Git Bash. Continúa la guía de 17 páginas sin sustituir sus instrucciones previas.

1. HEAD, master/main y origin

HEAD señala la posición actual de trabajo. Si apunta a master, cada nuevo commit avanza esa rama. main y master son nombres habituales de una rama local, sin diferencias técnicas. origin es el nombre del remoto, no una rama.

```
HEAD -> master -> 38f40ce
origin/master -> 38f40ce
```

origin/master refleja la última posición conocida de master en GitHub; no consulta el servidor continuamente. origin/HEAD indica localmente cuál es la rama predeterminada del remoto, por ejemplo origin/master. No equivale a tu HEAD local.

2. Checkout de un commit antiguo (detached HEAD)

En Lazygit: panel [4] Commits; seleccionar abd (d0618e9); Espacio; elegir Checkout commit ... as detached head y confirmar. HEAD apunta directamente a ese commit, pero la rama master permanece donde estaba.

```
HEAD -> d0618e9 (abd)
master -> 38f40ce
```

Para regresar: panel [3] Local branches, seleccionar master y pulsar Espacio. El cambio de versión modifica los archivos rastreados en disco; un archivo ya abierto en Neovim puede mostrar todavía el contenido que tenía en memoria. Si no hay ediciones sin guardar, :e! recarga el archivo desde el disco (descarta las ediciones locales no guardadas).

3. Ramas: crear, cambiar, consultar historial

En detached HEAD, desde Local branches se puede crear una rama con n (New branch) y darle un nombre, como commit_02_branch. Una rama es una referencia a un commit y a todos sus antecesores; varias ramas pueden apuntar al mismo commit. En [3], seleccionar una rama para consultar su log y Espacio para cambiar a ella. Para regresar de vistas secundarias, Esc.

Recuperar versiones sin perder historial

4. Checkout, Reset y Restore + Commit

Checkout: permite mirar un commit anterior sin modificar la punta de master. Reset: mueve la rama actual al commit seleccionado. Restore + Commit: devuelve los archivos versionados al contenido de un commit anterior, pero registra esa restauración como un commit nuevo y conserva los commits intermedios en master.

5. Soft, Mixed y Hard Reset

Soft: mueve la rama; deja los cambios posteriores preparados en staging. Mixed: mueve la rama y deja los cambios posteriores sin preparar, conservando el contenido de trabajo. Hard: mueve la rama, ajusta staging y restaura los archivos rastreados al commit elegido: puede descartar trabajo. En Lazygit, panel Commits, seleccionar commit y pulsar g para Reset.

Los commits posteriores dejan de pertenecer al historial de la rama tras un Reset, aunque pueden permanecer accesibles en otras referencias o temporalmente mediante Reflog. Si ya se publicaron, un push normal puede rechazarse. Evitar force push sobre trabajo compartido.

6. Recuperar abd como nueva última versión

Método recomendado para recuperar una versión anterior conservando todos los commits en el historial. Desde Git Bash, estando en master y con el árbol de trabajo limpio:

```
git switch master
git status
git restore --source=d0618e9 --staged --worktree :/
```

Luego abrir Lazygit, revisar los cambios preparados, crear un nuevo commit con c y subirlo con Shift+P. También se puede completar enteramente en Git Bash con git commit -m "Restaurar version abd" y git push. Los archivos no rastreados no se borran automáticamente.

Precaución: restore sobrescribe cambios de archivos rastreados: guardar o respaldar el trabajo previo. No es equivalente a un Hard Reset.

7. Reflog y Stash

Reflog (pestaña del panel [4]) registra movimientos de HEAD, checkouts y resets; ayuda a recuperar referencias a commits. Stash guarda temporalmente cambios sin crear commits: desde Files usar s; desde [5] Stash se recuperan con Apply o Pop. Por defecto no guarda archivos nuevos sin seguimiento; pueden aparecer conflictos al aplicar.

GitHub y rutina de trabajo

8. Fetch, Pull, Push y Status

Fetch: actualiza las referencias remotas sin integrar cambios en master ni modificar el directorio de trabajo. Pull: integra cambios remotos. Push: publica los commits locales. Status: muestra la rama y el estado de los archivos. Lazygit realiza fetch automático a intervalos; el mensaje Fetching... es normal y no significa que altere archivos.

En la configuración probada, con el panel [3] Local branches: p para Pull y Shift+P para Push. Los atajos dependen del panel; consultar ?. Un solo Push publica todos los commits pendientes de la rama.

9. Flechas de estado y sincronización

↑3 significa tres commits locales aún no presentes en la rama remota de seguimiento; ↓2 significa dos commits remotos conocidos pendientes de integrar. ✓ indica que la rama está alineada con su referencia de seguimiento conocida. Los cambios de archivos se ven aparte en Files.

```
git status -sb
## master...origin/master
```

La salida anterior sin ahead/behind ni archivos listados indica árbol limpio y sincronización con la referencia remota conocida. Para comprobar cambios remotos nuevos, antes ejecutar git fetch.

10. Primera publicación y upstream

Si origin/master todavía no existe, configurar manualmente upstream puede mostrar el error «upstream branch origin/master not found». En Lazygit, Shift+P permite publicar la rama. Alternativa desde Git Bash:

```
git push -u origin master
```

Después el log puede mostrar HEAD → master, origin/master y origin/HEAD juntos sobre el mismo commit. Crear una rama local no la publica automáticamente en GitHub.

11. Atajos y precauciones

Espacio, g, g: abrir Lazygit desde Neovim cuando Espacio es mapleader. Espacio en Files: preparar o retirar archivo. c: crear commit en el panel correspondiente. 4: panel Commits. 3: Local branches. q: salir. Esc: retroceder de un subpanel. ?: consultar teclas.

En la prueba se versionaron archivos de respaldo LaTeX con sufijo ~. Conviene incluirlos en .gitignore en proyectos futuros. Agregar una regla .gitignore no deja de rastrear automáticamente los archivos que ya están registrados.